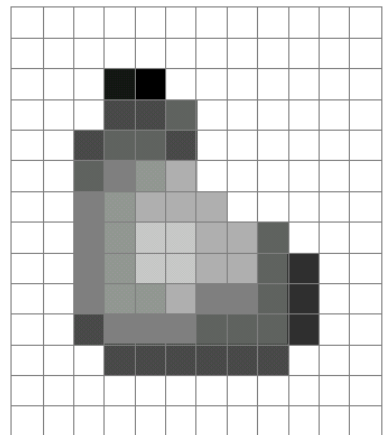


Image filters

Repetition: What is an image?

- A grid (matrix) of intensity values



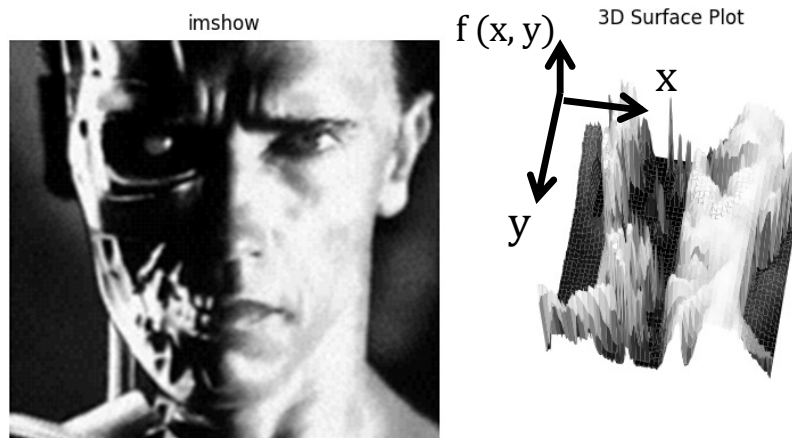
=

255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	20	0	255	255	255	255	255	255	255
255	255	255	75	75	75	255	255	255	255	255	255
255	255	75	95	95	75	255	255	255	255	255	255
255	255	96	127	145	175	255	255	255	255	255	255
255	255	127	145	175	175	175	255	255	255	255	255
255	255	127	145	200	200	175	175	95	255	255	255
255	255	127	145	200	200	175	175	95	47	255	255
255	255	127	145	145	175	127	127	95	47	255	255
255	255	74	127	127	127	95	95	95	47	255	255
255	255	255	74	74	74	74	74	74	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255

(common to use one byte per value: 0 = black, 255 = white)

What is an image?

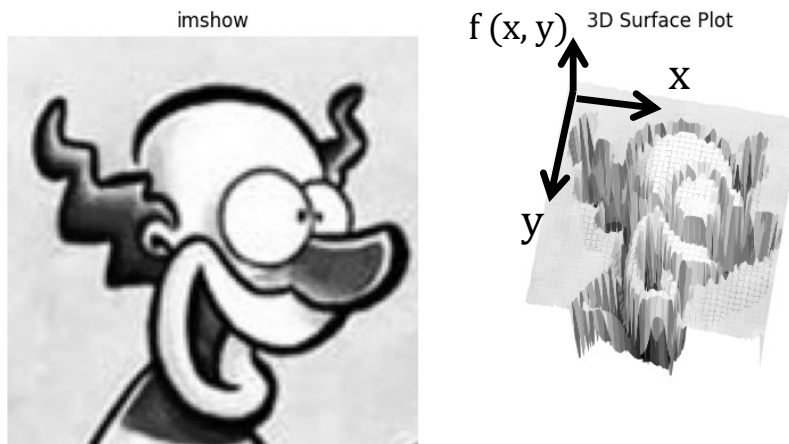
- Can think of a (grayscale) image as a **function** f from $\mathbb{R}^2$ to $\mathbb{R}$:
- $f(x, y)$ gives the **intensity** at position (x, y)



- A **digital** image is a discrete (**sampled, quantized**) version of this function

What is an image?

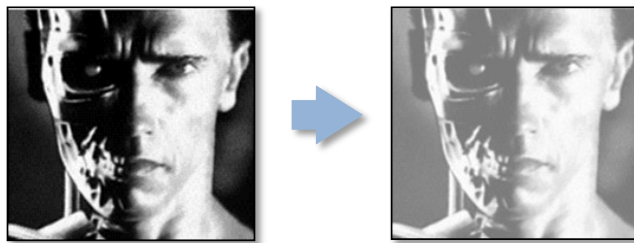
- Can think of a (grayscale) image as a **function** f from $\mathbb{R}^2$ to $\mathbb{R}$:
- $f(x, y)$ gives the **intensity** at position (x, y)



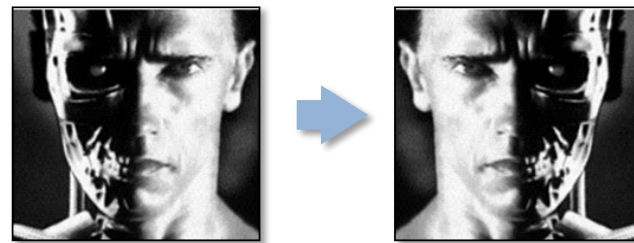
- A **digital** image is a discrete (**sampled, quantized**) version of this function

Image transformations

- As with any function, we can apply operators to an image



$$g(x, y) = f(x, y) + 20$$



$$g(x, y) = f(-x, y)$$

- Today we'll talk about a special kind of operator, *convolution* (linear filtering)

Filters

Filtering

- Form a new image whose pixel values are a combination of the original pixel values
- Why?
 - To get useful information from images
 - **E.g., extract edges or contours (to understand shape)**
 - To enhance the image
 - **E.g., to remove noise**
 - **E.g., to sharpen and “enhance image”**
- A key operator in Convolutional Neural Networks

Canonical Image Processing problems

- Image Restoration
 - denoising
 - deblurring
- Image Compression
 - JPEG, HEIF, MPEG, ...
- Locating Structural Features
 - corners
 - edges

Question: Noise reduction

- Given a camera and a still scene, how can you reduce noise?



Take lots of images and average them!

Source: S. Seitz

Image filtering

- Modify the pixels in an image based on some function of a local neighborhood of each pixel

10	5	3
4	5	1
1	1	7

Local image data

Some function

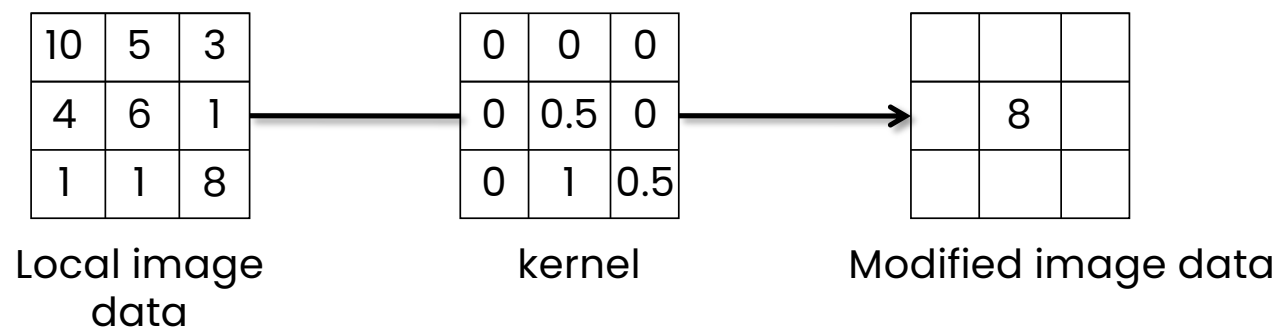


	7	

Modified image data

Linear filtering

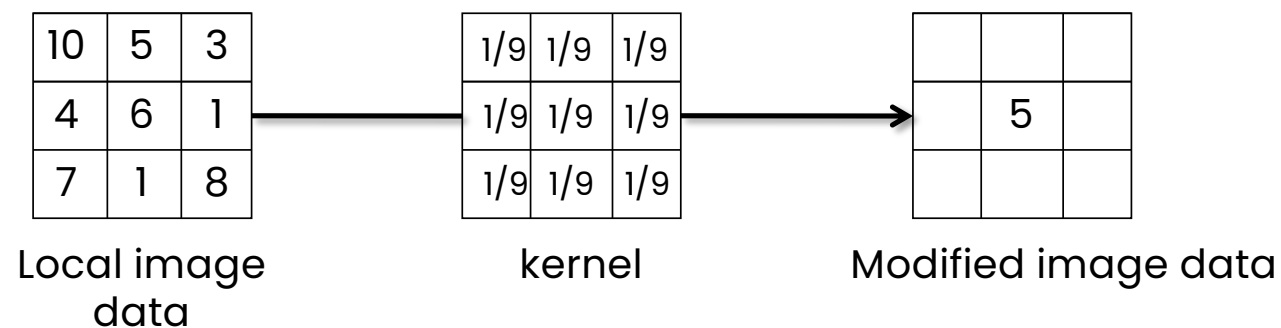
- One simple version of filtering: linear filtering (cross-correlation, convolution)
- Replace each pixel by a linear combination (a weighted sum) of its neighbors
- The prescription for the linear combination is called the “kernel” (or “mask”, “filter”)



Source: L. Zhang

Mean filtering

- To compute the mean, we simply set each kernel value to $1/N$ where N is the size of the kernel.



Source: L. Zhang

Cross-correlation

- Let F be the image, H be the kernel (of size $2k+1 \times 2k+1$), and G be the output image

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

This is called a **cross-correlation** operation:

$$G = H \otimes F$$

Can think of as a “dot product” between local neighborhood and kernel for each pixel

Convolution

- Same as cross-correlation, except that the kernel is “flipped” (horizontally and vertically)

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

This is called a **convolution** operation:

$$G = H * F$$

- Convolution is **commutative** and **associative**

Math vs Code

 **Freya Holmér** ✓
@FreyaHolmer

btw these large scary math symbols are just for-loops

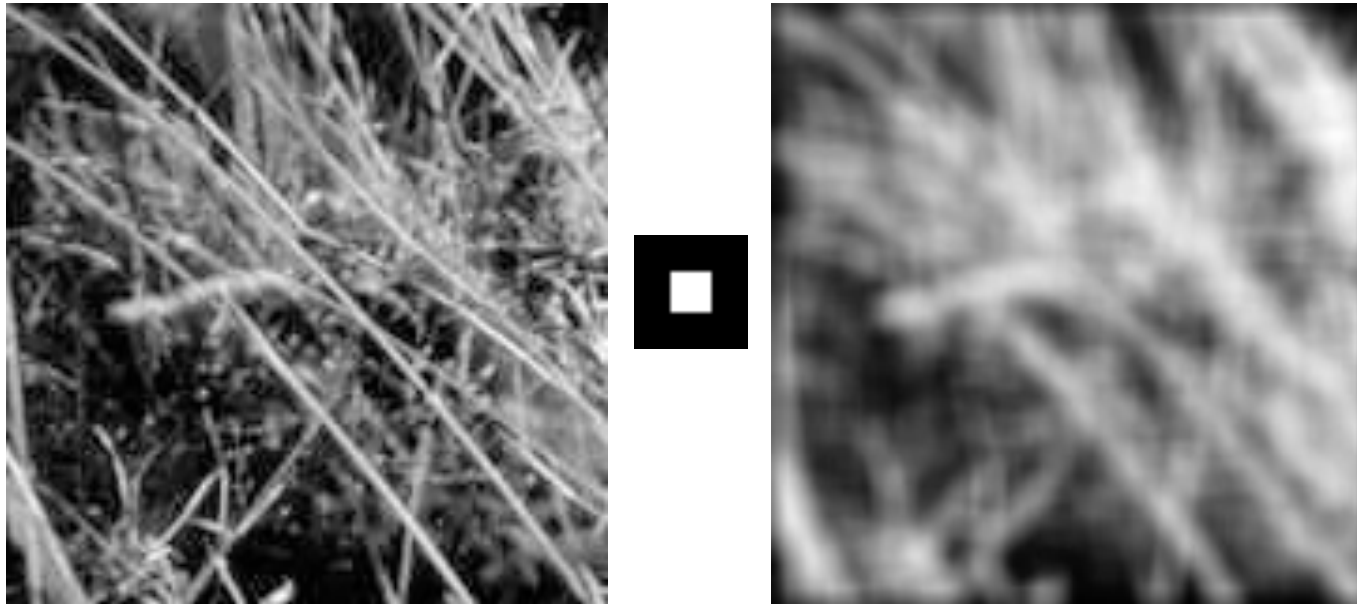
[Post übersetzen](#)

Summation (capital sigma)	$\sum_{n=0}^4 3n$	<pre>sum = 0; for(n=0; n<=4; n++) sum += 3*n;</pre>
Product (capital pi)	$\prod_{n=1}^4 2n$	<pre>prod = 1; for(n=1; n<=4; n++) prod *= 2*n;</pre>

4:21 nachm. · 11. Sep. 2021

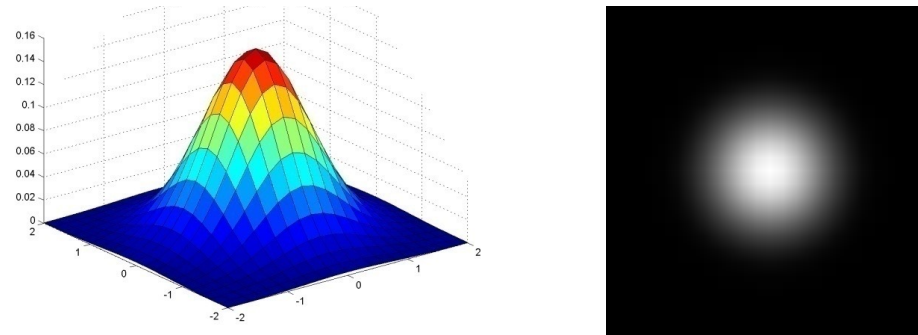
570 7.864 36.385 5.027 

Smoothing with mean (aka box) filter revisited



Source: D. Forsyth

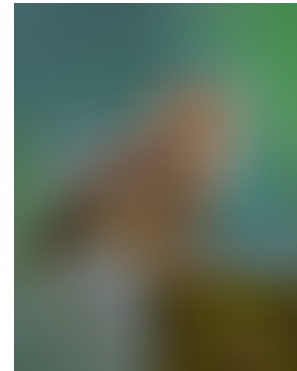
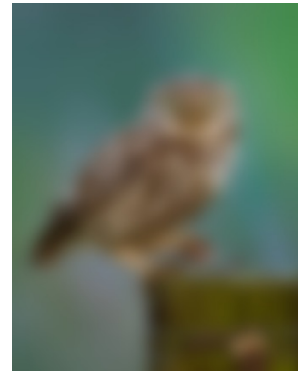
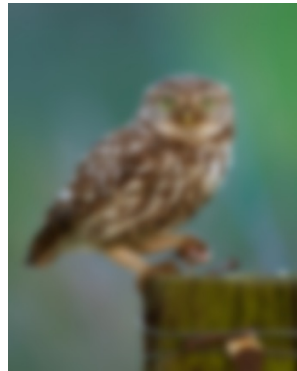
Gaussian kernel



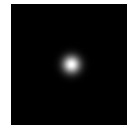
$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Source: C. Rasmussen

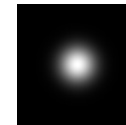
Gaussian filters



$\sigma = 1$ pixel



$\sigma = 5$ pixels



$\sigma = 10$ pixels



$\sigma = 30$ pixels

Gaussian filter

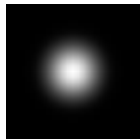
- Removes “high-frequency” components from the image
 - Hence called a low-pass filter
- Convolution with self is another Gaussian



Mean vs. Gaussian filtering

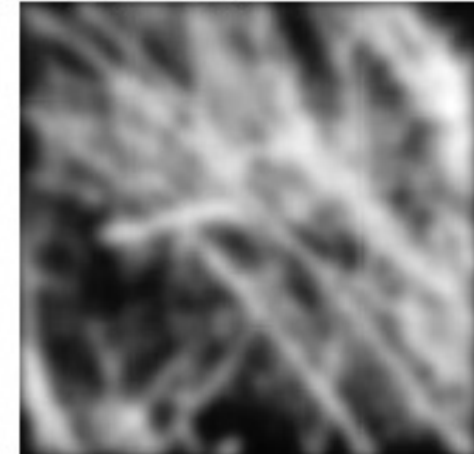
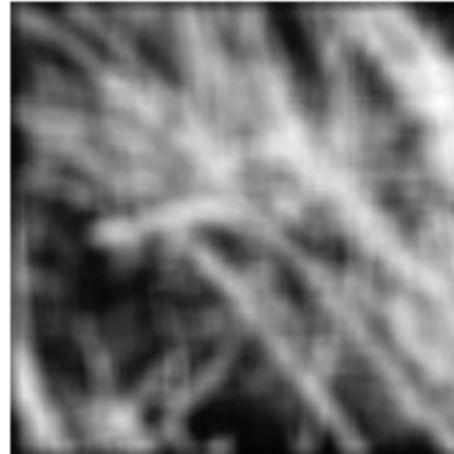


Mean/box filter

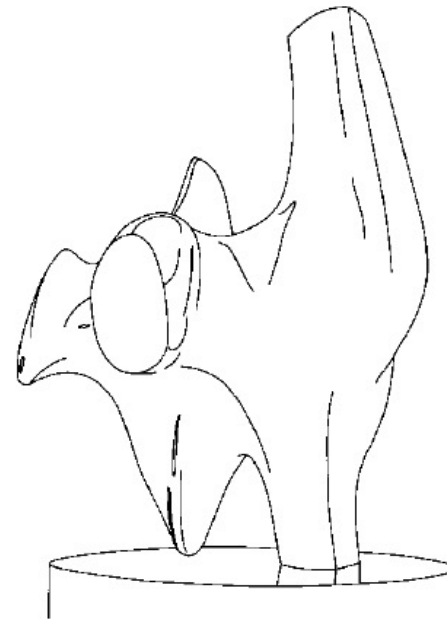
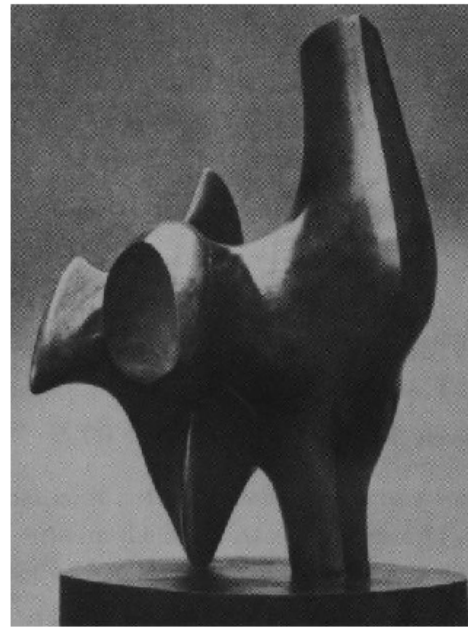


Gaussian filter

Gaussian filter leads to smoother results.

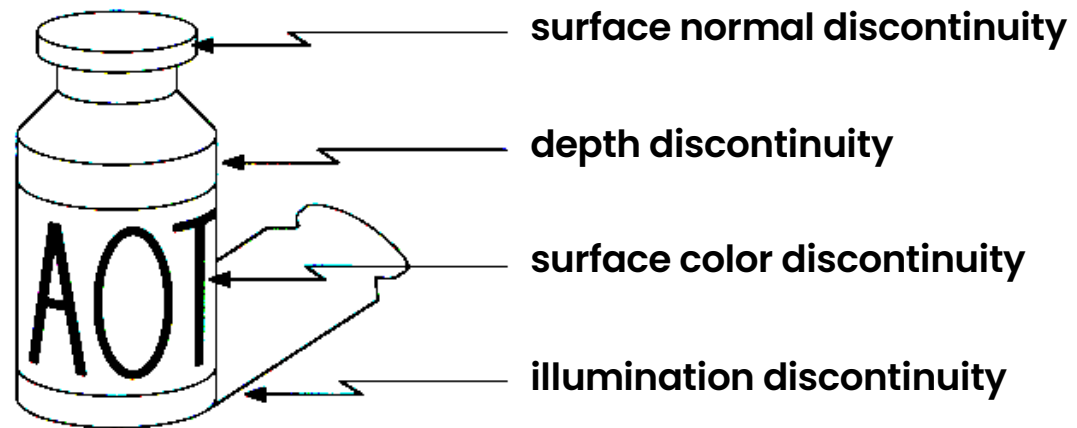


Edge detection



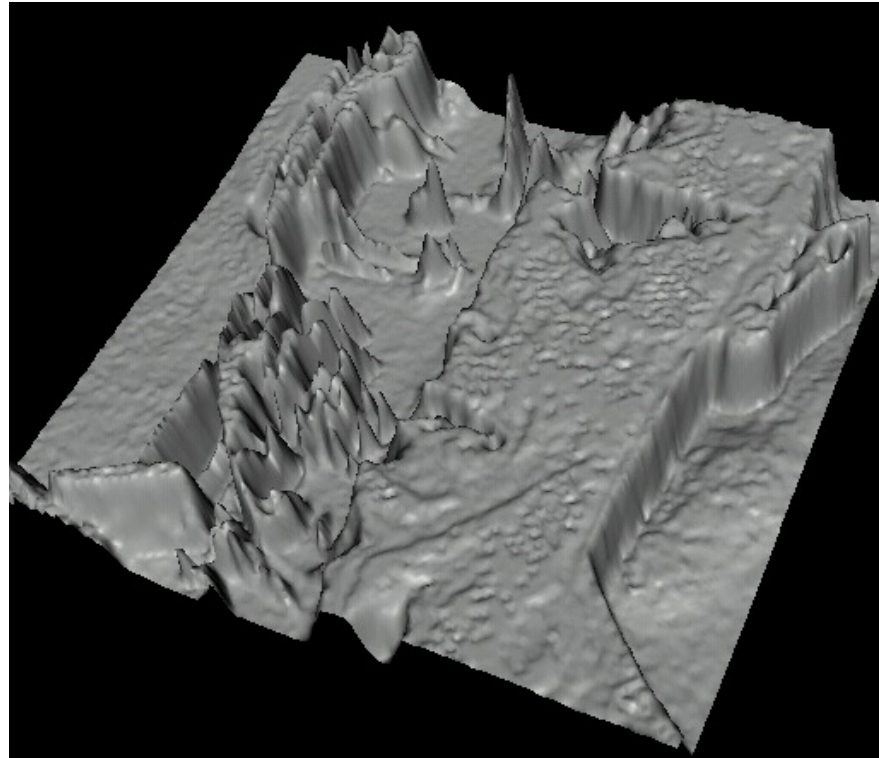
- Convert a 2D image into a set of curves
 - Extracts salient features of the scene
 - More compact than pixels

Origin of edges



- Edges are caused by a variety of factors

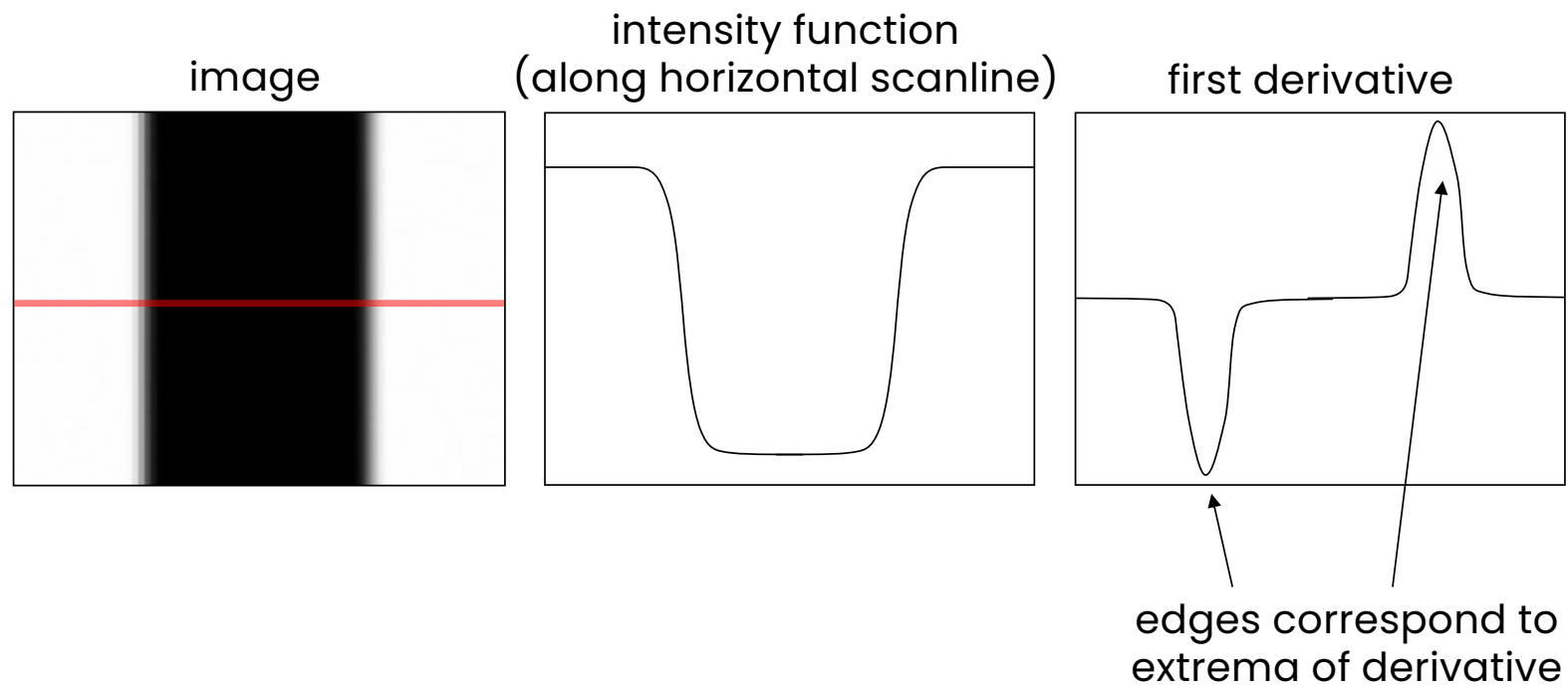
Images as functions...



- Edges look like steep cliffs

Characterizing edges

- An edge is a place of *rapid change* in the image intensity function



Source: L. Lazebnik

Image derivatives

- How can we differentiate a *digital* image $F[x, y]$?
 - Option 1: reconstruct a continuous image f , then compute the derivative
 - Option 2: take discrete derivative (finite difference)

$$\frac{\partial f}{\partial x}[x, y] \approx F[x + 1, y] - F[x, y]$$

This can be easily implemented as a linear filter!

$$\frac{\partial f}{\partial x} : \begin{array}{|c|c|c|} \hline & & \\ \hline 1 & -1 & \\ \hline & & \\ \hline \end{array}$$

H_x

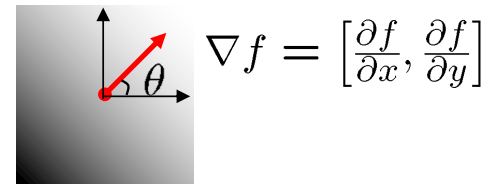
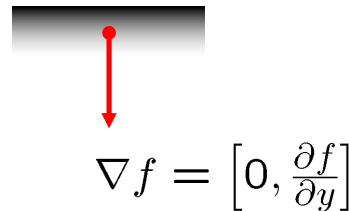
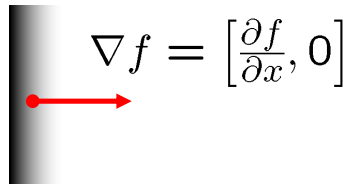
$$\frac{\partial f}{\partial y} : \begin{array}{|c|c|c|} \hline & & \\ \hline & -1 & \\ \hline & 1 & \\ \hline \end{array}$$

H_y

Image gradient

- The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$

The gradient points in the direction of most rapid increase in intensity

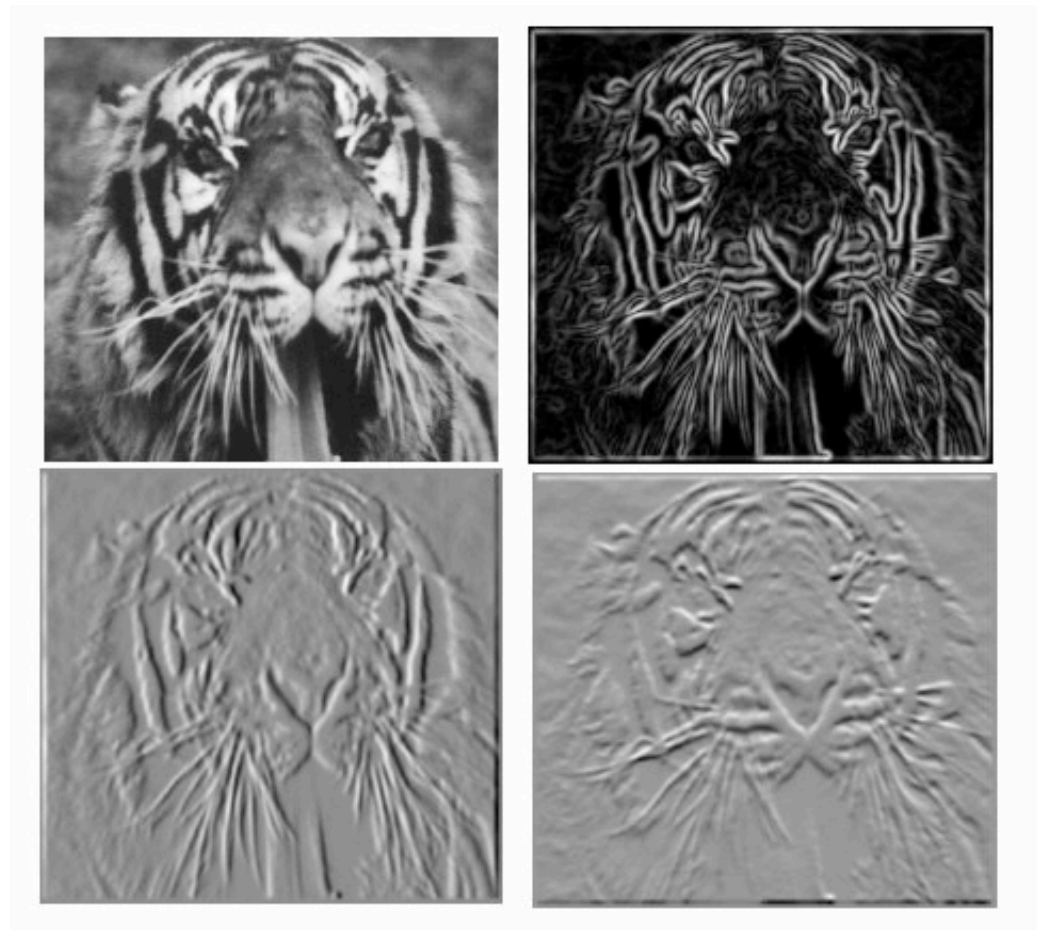


The *edge strength* is given by the gradient magnitude: $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$

The gradient direction is given by: $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

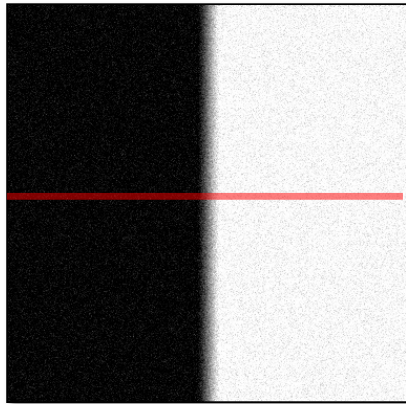
how does this relate to the direction of the edge?

Image gradient



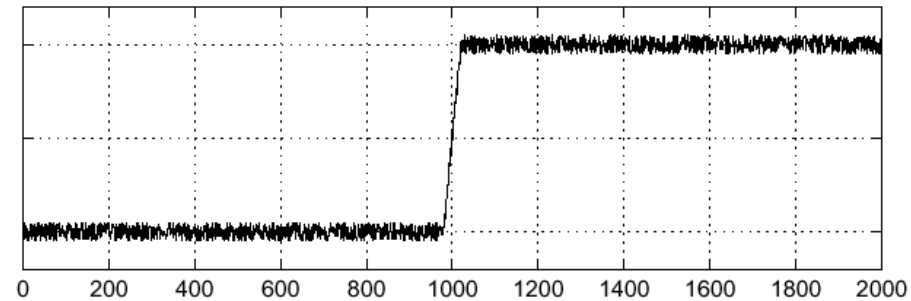
Source: L. Lazebnik

Effects of noise

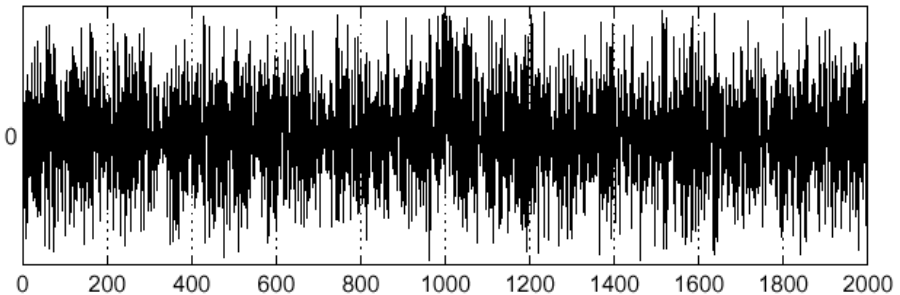


Noisy input image

$$f(x)$$



$$\frac{d}{dx}f(x)$$



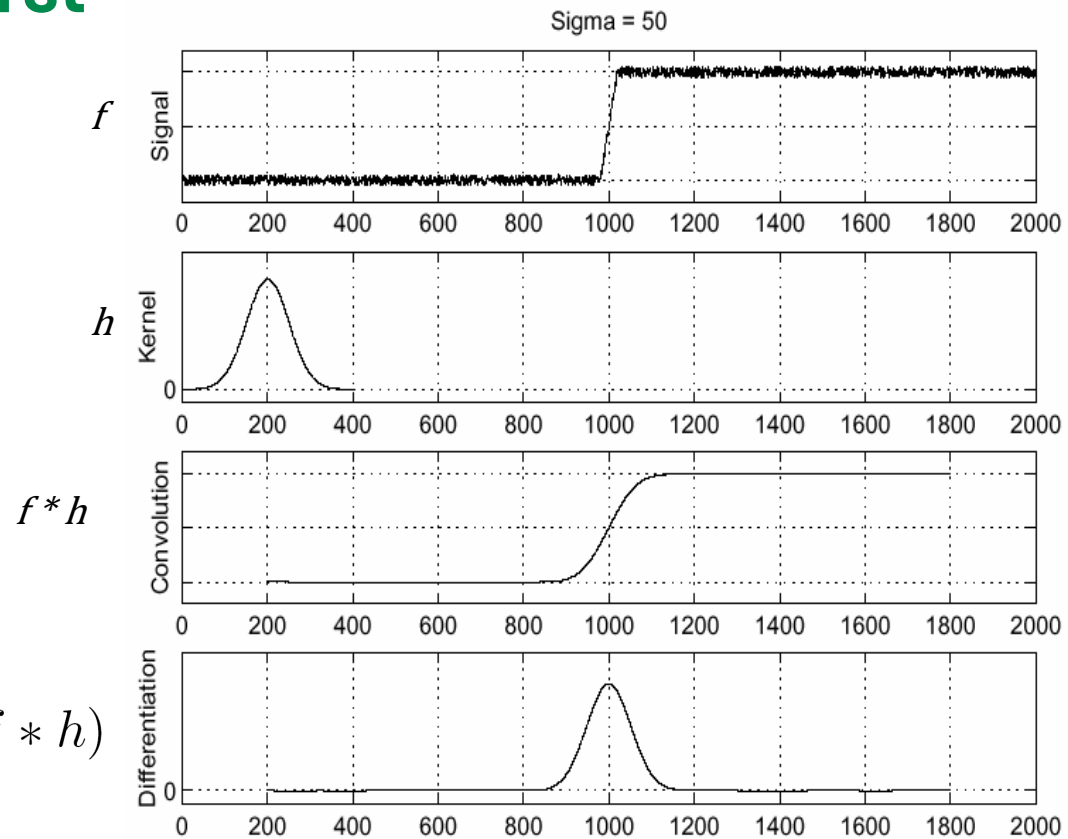
Where is the edge?

Source: S. Seitz

Solution: smooth first

- To find edges, look for peaks in $\frac{d}{dx}(f * h)$

$$\frac{d}{dx}(f * h)$$



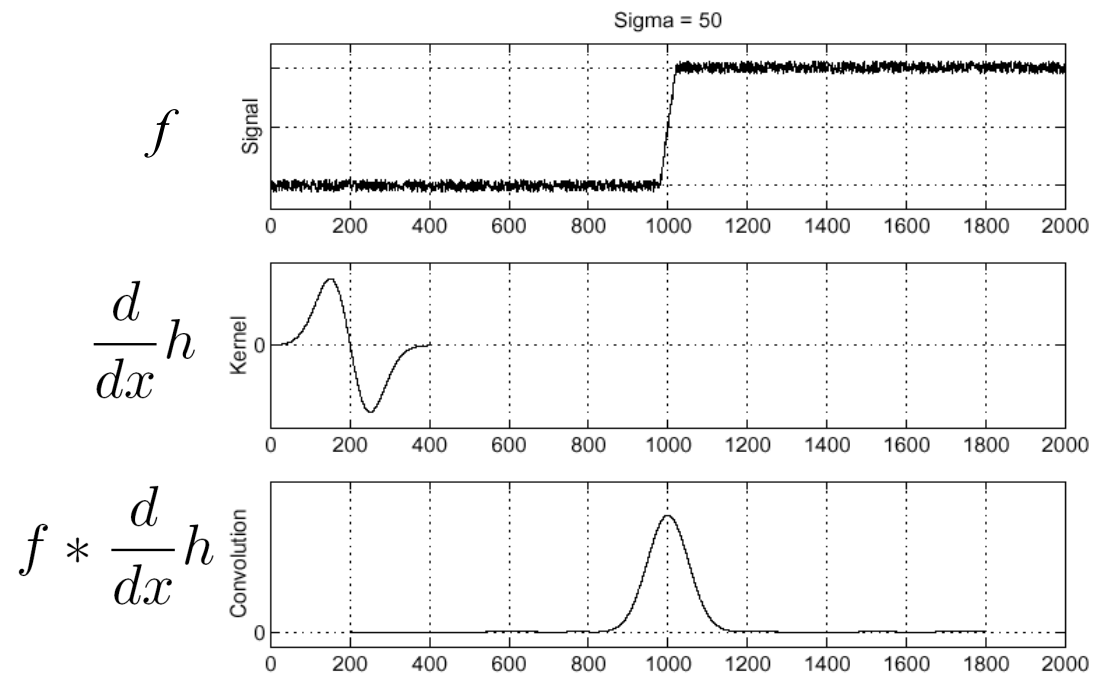
Source: S. Seitz

Associative property of convolution

Differentiation is convolution, and convolution is associative:

$$\frac{d}{dx}(f * h) = f * \frac{d}{dx}h$$

- This saves us one operation:



Source: S. Seitz

The Sobel operator

- Common approximation of derivative of Gaussian

$$\frac{1}{8} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

s_x

$$\frac{1}{8} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

s_y

- The standard definition of the Sobel operator omits the $1/8$ term
 - doesn't make a difference for edge detection
 - the $1/8$ term **is** needed to get the right gradient magnitude

Canny edge detector

1. Filter image with derivative of Gaussian
2. Find magnitude and orientation of gradient
3. Non-maximum suppression
4. Linking and thresholding (hysteresis):
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them

OpenCV: `cv2.Canny(image, threshold1=50, threshold2=150)`



Source: D. Lowe, L. Fei-Fei, J. Redmon

Filters: Thresholding



$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \text{otherwise} \end{cases}$$

Morphological operations

- Morphological operations apply a structuring element to an input image and generate an output image.
- A structuring element can be of any shape; a filter kernel is typically square.
- **Dilation**: logical **OR** or max in OpenCV*
- **Erosion**: logical **AND** or min in OpenCV*



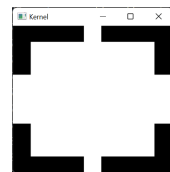
* https://docs.opencv.org/4.x/db/df6/tutorial_erosion_dilatation.html

Morphological operations

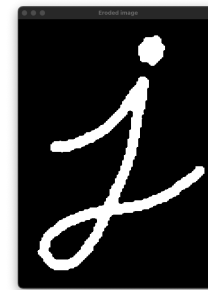
- What happens on noisy images?



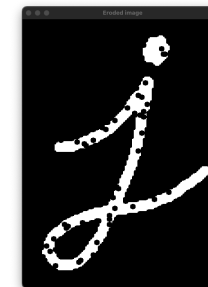
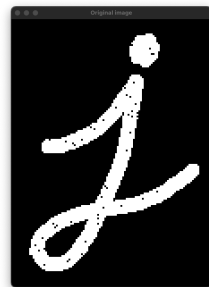
Original



Dilation



Erosion

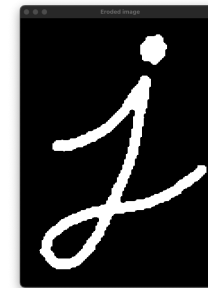


Morphological operations

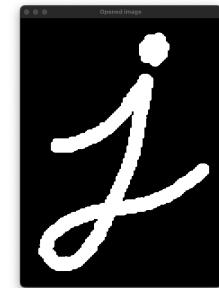
- **Open:** First erosion, then dilatation.
- **Close:** First dilation, then erosion.



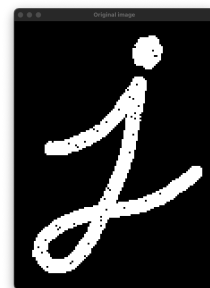
Original



Erosion



Open



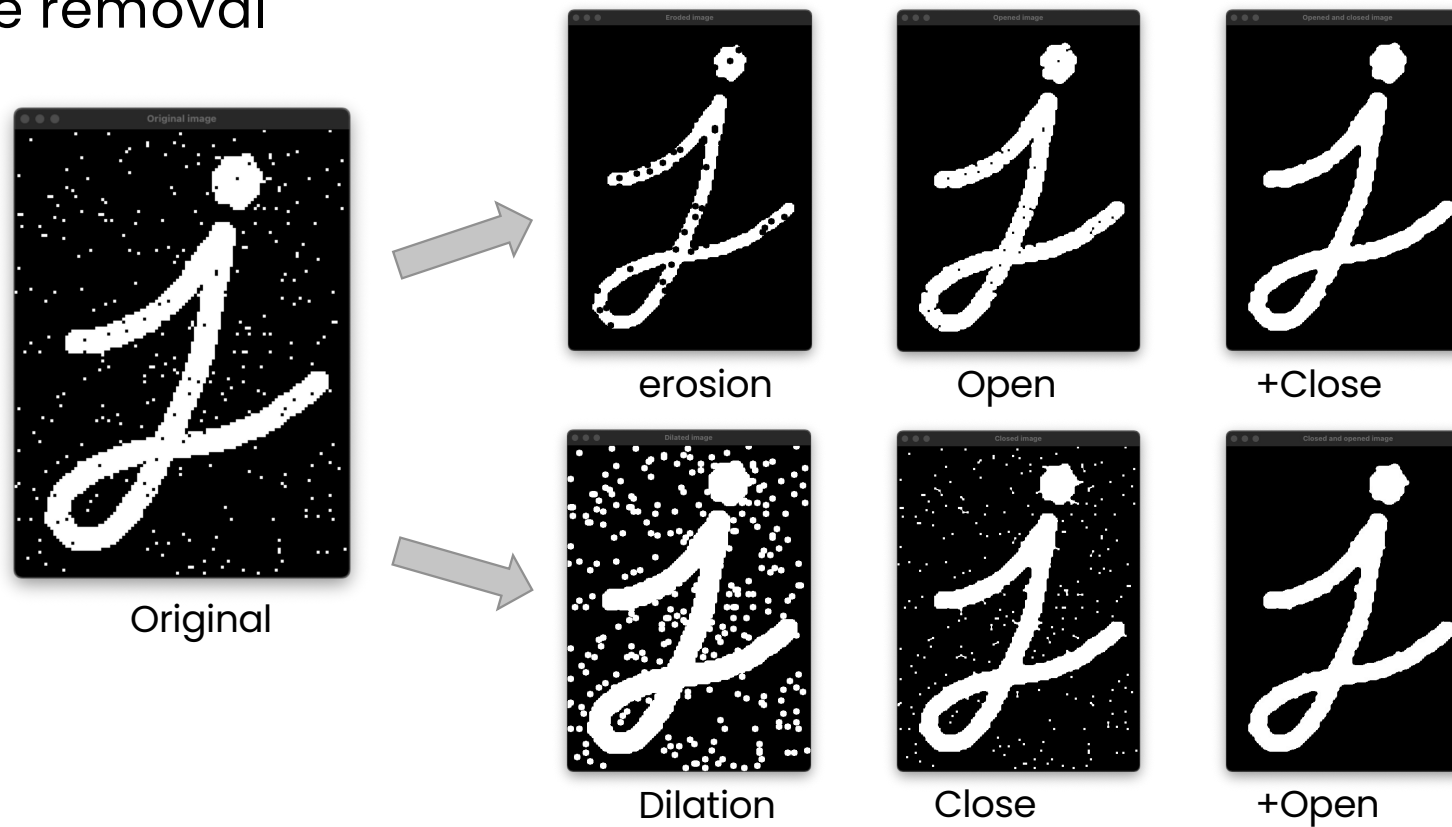
Dilation



Close

Morphologische Operationen

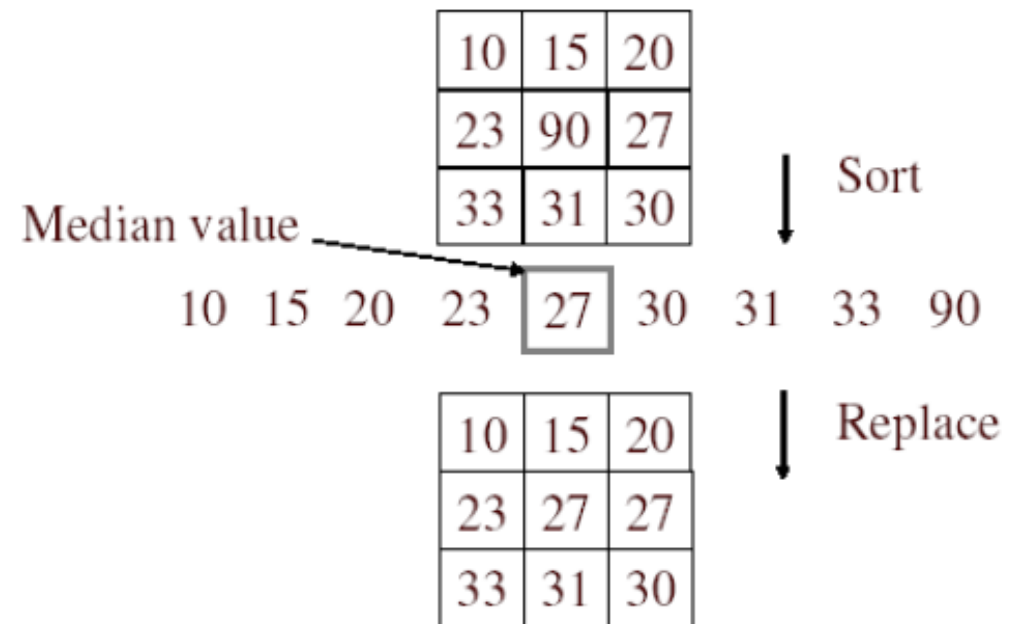
- Noise removal



Median Filter

A nice filter to remove noise.

- No new color values are created
- Smooths peaks, good for salt'n'pepper noise



Linear filters?

- Can thresholding be implemented with a linear filter?
- Are the morphological operations linear filters?
- Is the median filter linear?

Nope. Hence, be careful when combining them.